

Hash Tables: $O(1)$ Average, By Giving Up Order

Programming and Data Structures — Week 8, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain the core hash table idea of converting an arbitrary key into an array index directly, without searching or comparing against other keys; implement a simple hash function and explain the properties that make a hash function “good”; explain precisely what a collision is, why collisions are mathematically unavoidable, and trace at least one collision-resolution strategy completely by hand; and state and rigorously justify the average-case $O(1)$ cost claim for hash table operations, together with its important worst-case caveat.

Outline

Today covers a coat-check-ticket analogy motivating direct-index access for arbitrary keys; the hash function, which converts a key into an array index; the properties that make a hash function good; collisions, why they are mathematically unavoidable regardless of hash function quality; chaining as a collision-resolution strategy, traced completely; a rigorous justification of the average-case $O(1)$ cost claim, together with its worst-case caveat; the MTech-track alternative of open addressing; an in-class quiz; and a summary.

The coat-check-ticket analogy

Picture a coat-check counter at a theater: handing over a coat produces a numbered ticket, and retrieving the coat later means the attendant reads the ticket’s number and walks directly to that numbered hook, never needing to search through every coat on the rack. A hash table provides exactly this capability for arbitrary keys that are not naturally small integers — a string like `"alice@example.com"`, for instance — by converting the key into a numbered “hook,” an array index, directly and predictably.

From key to index: the hash function

```
def simple_hash(key, table_size):  
    total = sum(ord(ch) for ch in key)    # sum of character codes  
    return total % table_size             # fit into the table's range
```

A hash function converts any key — a string, a tuple, or in general any object — into an integer index that fits within the underlying array's bounds. The modulo operation here, `% table_size`, is the exact same technique used in Week 3 Lecture 3's circular buffer, mapping an arbitrarily large range of possible sums down to a small, fixed range of valid array indices. A single, non-negotiable requirement governs every hash function: given the same key twice, it must produce the exact same index both times — otherwise a value stored under a key could never be reliably found again.

Tracing the hash function on real keys

```
simple_hash("cat", 10):  
    ord('c')=99, ord('a')=97, ord('t')=116 → sum=312  
    312 % 10 = 2  
  
simple_hash("act", 10):  
    ord('a')=97, ord('c')=99, ord('t')=116 → sum=312  
    312 % 10 = 2    <-- SAME index as "cat"! (an anagram problem, examined next)
```

Tracing this simple hash function on two keys reveals a real weakness: "cat" and "act", being anagrams of each other, produce the identical character-code sum, 312, and therefore the identical hash index, 2 — despite being entirely different strings. This is not a bug in the modulo step, but a genuine flaw in the hash function's design itself: summing character codes discards all information about their *order*, causing every anagram of a given string to collide unavoidably. This motivates the next slide's discussion of what makes a hash function genuinely good.

What makes a hash function “good”

A good hash function satisfies three properties. It must be deterministic, as already established. It should distribute different keys as uniformly as possible across the available indices, avoiding clustering that would concentrate many keys onto the same few indices. And it must be fast to compute, since the entire purpose of a hash table is avoiding a search — a hash function that is itself slow to evaluate undermines that goal. Real-world hash functions, including Python's built-in `hash()` for strings, incorporate each character's *position* into the computation specifically to avoid the anagram-collision problem demonstrated on the previous slide.

Collisions: unavoidable, and why

A collision occurs when two different keys hash to the same array index. Collisions are mathematically unavoidable in general: by the pigeonhole principle, if there are more possible keys than available table slots — which is true for essentially every realistic hash table, since string keys vastly outnumber any practical table size — at least one collision must occur for *some* pair of keys, regardless of how carefully the hash function is designed. This reframes the entire remaining design problem: the question a hash table implementation must answer is never “how do we prevent collisions” but “how do we handle them correctly and efficiently when they occur.”

Resolving collisions: chaining

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.buckets = [[] for _ in range(size)]    # each slot holds a LIST

    def put(self, key, value):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.buckets[index]):
            if k == key:
                self.buckets[index][i] = (key, value)    # update existing
                return
        self.buckets[index].append((key, value))        # append new

    def _hash(self, key):
        return sum(ord(ch) for ch in str(key)) % self.size
```

Chaining resolves collisions by making each array slot, called a bucket, hold a list of key-value pairs rather than a single value. When two keys collide, hashing to the same index, they simply coexist in the same bucket’s list, distinguished by their actual key value stored alongside each entry. Retrieving a value requires computing the hash to find the correct bucket, then scanning that bucket’s list to find the entry whose key actually matches — this scan is precisely the cost that a collision introduces, and it is bounded by how many keys happen to share that bucket.

Tracing chaining completely, with a collision

```
HashTable(size=5). hash("cat")=? ord sum 312 % 5 = 2. hash("dog")? sum=314 % 5 = 4.
hash("bat")? sum=314 % 5 = 4    <-- collides with "dog"!
```

```

put("cat", 1): buckets[2] = [("cat", 1)]
put("dog", 2): buckets[4] = [("dog", 2)]
put("bat", 3): buckets[4] = [("dog", 2), ("bat", 3)]    <-- appended, collision resolved

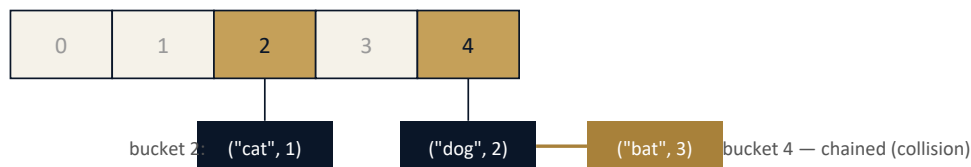
get("bat"): hash("bat")=4 → scan buckets[4]: ("dog",2) no match, ("bat",3) MATCH → return 3

```

Tracing three insertions where two keys, "dog" and "bat", happen to collide at index 4: "cat" lands cleanly at index 2 with no collision. "dog" lands at index 4. "bat" also hashes to index 4, colliding with "dog" — chaining resolves this simply by appending "bat"'s entry to the same bucket's list, alongside "dog"'s. Retrieving "bat" requires computing its hash (4), then scanning that bucket's list: first checking "dog" (no match), then checking "bat" (match), returning its value. The collision cost exactly 2 comparisons instead of 1.

The picture: buckets, some empty, some chained

Bucket array (size 5)



Buckets 0, 1, 3 are empty. Bucket 2 holds one entry. Bucket 4 holds a chain of two entries that collided on the same hash index.

This diagram makes the chaining structure concrete: most buckets hold exactly one entry, requiring no scanning at all, while the one bucket that experienced a collision holds a short chained list, requiring a brief scan proportional to how many keys happen to share that bucket.

Why average-case $O(1)$ — the load factor argument

The load factor, $\alpha = n/\text{size}$, is the average number of entries per bucket if keys were spread perfectly evenly. With a good hash function and the load factor kept small — typically by resizing the underlying bucket array once it grows too full, directly reusing Week 2's dynamic array doubling strategy — the average chain length any single lookup must scan stays $O(1)$, a small constant independent of n . The

total average-case cost of search, insert, or delete is therefore the $O(1)$ cost to compute the hash, plus the $O(1)$ average chain scan, giving $O(1)$ overall — the headline result this lecture has been building toward.

The worst-case caveat, stated honestly

It is essential to be honest about the worst case. If every key happens to hash to the same single index — whether from a poorly designed hash function, or from an adversary deliberately choosing keys to attack a known hash function (a genuine security concern called “hash flooding” in real systems) — every entry collapses into one enormous chain in a single bucket, and every operation degrades to $O(n)$, no better than Week 4’s plain linked list, with all the hash table’s benefit entirely lost. This is structurally the same average-versus-worst-case distinction Week 6 Lecture 2 established for quicksort — a familiar shape of caveat, recurring here in a genuinely different structure.

MTech addition — open addressing, a different strategy

```
def put_open_addressing(self, key, value):
    index = self._hash(key)
    while self.slots[index] is not None and self.slots[index][0] != key:
        index = (index + 1) % self.size    # linear probing: try the NEXT slot
    self.slots[index] = (key, value)
```

Open addressing takes a fundamentally different approach: rather than allowing a bucket to hold multiple entries via a chained list, every slot in the array holds at most one entry directly. When a collision occurs, the algorithm “probes” forward — linear probing, shown here, simply tries the next slot, wrapping around via modulo exactly like Week 3 Lecture 3’s circular buffer — until an empty slot or a match is found. This trades chaining’s flexibility for better memory locality, since no separate list allocations are needed, at the cost of being more sensitive to a rising load factor (probe sequences grow longer as the table fills) and requiring more careful handling of deletion, since naively emptying a slot can break the probe sequence needed to find entries that come later.

Resizing a hash table: Week 2’s idea, applied again

When the load factor exceeds a chosen threshold, commonly around 0.7, a hash table resizes by allocating a new, larger bucket array, commonly double the previous size. Critically, every existing key must be re-hashed into this new array from scratch — the old computed indices are meaningless once `table_size` changes, since the hash function’s modulo step depends directly on it. This single resize operation costs $O(n)$, touching every stored entry, but exactly mirroring Week 2 Lecture 3’s amortized analysis of dynamic array doubling, this cost is spread, or amortized, across the many cheap

$O(1)$ insertions that occur between resizes — the same coin-jar and potential-function arguments from that lecture apply here essentially unchanged, applied to a different structure.

Exercise

As an exercise, implement `get(key)` and `delete(key)` for the chaining-based `HashTable`, tracing both by hand on the "dog"/"bat" collision example from this lecture, confirming deletion correctly removes only the targeted entry from its bucket's chain without disturbing any other entry. Separately, implement a position-aware hash function — multiplying a running total by a small prime number before adding each character's code, rather than simply summing — and confirm that "cat" and "act" no longer collide under this improved function.

Connecting back to Week 7's ordered-set trade-off

Week 7 Lecture 4 previewed exactly this trade-off ahead of time: a hash table achieves its $O(1)$ average performance precisely by abandoning any notion of key ordering. There is no structural relationship whatsoever between two keys' positions in the bucket array and their relative order as keys — "apple" and "banana" could hash to indices 7 and 2 respectively, or the reverse, with no predictable pattern. Consequently, a hash table cannot efficiently answer "what is the smallest stored key?" or "give me everything between X and Y?" without a full $O(n)$ scan of every bucket — this is not a limitation to be engineered away, but the direct and unavoidable structural price of scattering keys via a hash function to gain $O(1)$ average access in the first place.

Quiz — predict the collision

Using this lecture's original `simple_hash` function with table size 5, determine before checking the answer whether "ab" and "ba" collide, and explain precisely why.

Quiz — answer

Yes, "ab" and "ba" collide under this hash function, since addition is commutative: `ord('a') + ord('b')` equals `ord('b') + ord('a')` regardless of the characters' order, giving the identical sum and therefore the identical hash index. This confirms the general weakness identified earlier in this lecture: any two anagrams of each other will always collide under this specific character-summing hash function, no matter which two anagram strings are chosen — a structural flaw in the hash function's design, not a coincidence of any particular example.

Real-world use: Python's dict

Python's built-in `dict` type is a hash table, implemented essentially as described in this lecture, using open addressing internally since CPython 3.6. Every `dict[key] = value` assignment, `dict[key]` lookup, and `key in dict` membership check is this lecture's hash-then-resolve mechanism running directly underneath the familiar syntax. Understanding this explains real, observable Python behavior: why dictionary keys must be hashable, requiring a consistent `__hash__` method (mutable types like lists cannot be dict keys, precisely because their hash would need to stay constant even as their contents change); and why dictionary iteration order was historically unspecified before Python 3.7 — an implementation detail of exactly the bucket-array structure this lecture built from scratch.

Summary

A hash function converts an arbitrary key into an array index, enabling direct access to stored data without searching or comparing against other keys. Collisions — two keys hashing to the same index — are mathematically unavoidable by the pigeonhole principle, for any hash function, which reframes hash table design around handling collisions well rather than attempting to prevent them entirely. Chaining resolves collisions using per-bucket lists; open addressing resolves them by probing to alternative slots directly within the array. Given a good hash function and a load factor kept under control, average-case cost for search, insert, and delete is $O(1)$ — but the worst case, when many or all keys collide, remains $O(n)$, the same average-versus-worst-case caveat seen with quicksort in Week 6. Next lecture introduces heaps, a structure offering an entirely different kind of guarantee, centered on efficient priority-based processing rather than key-based lookup.